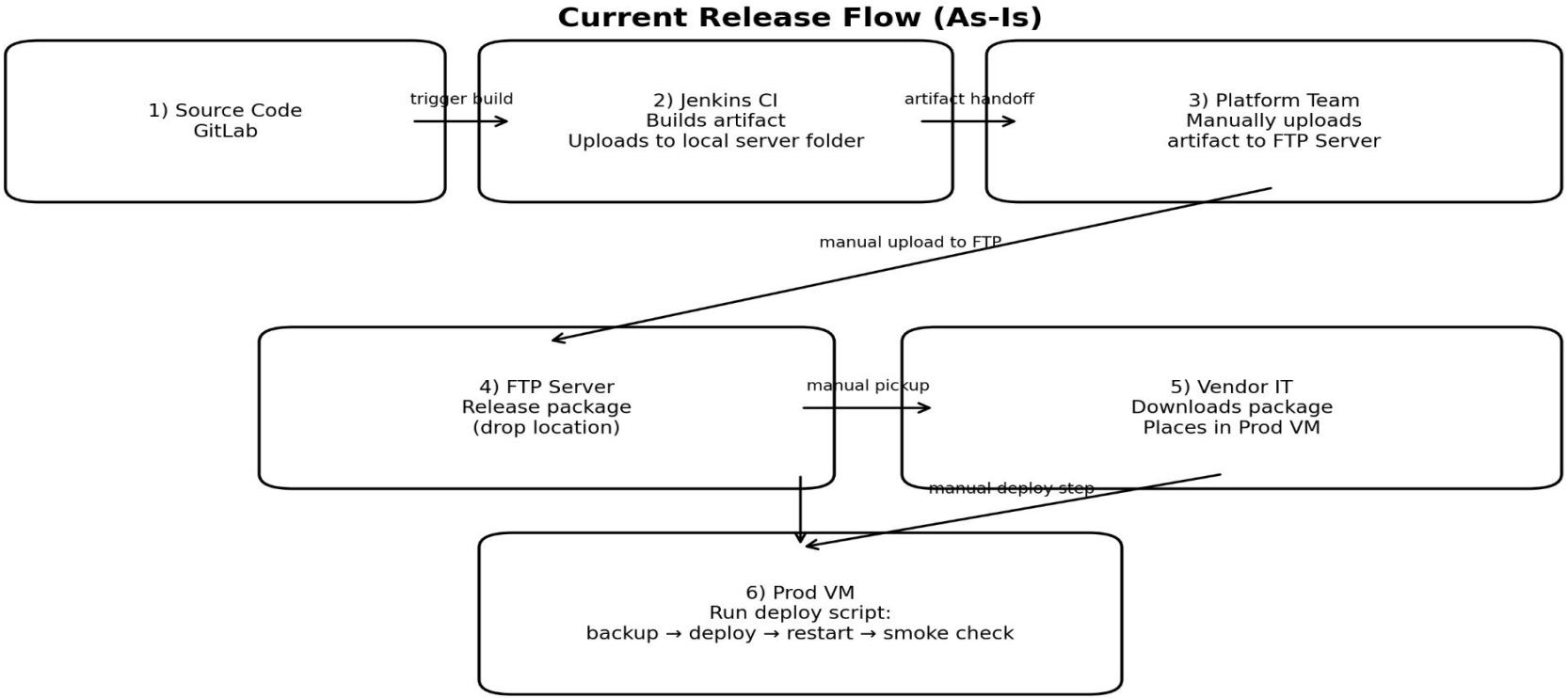
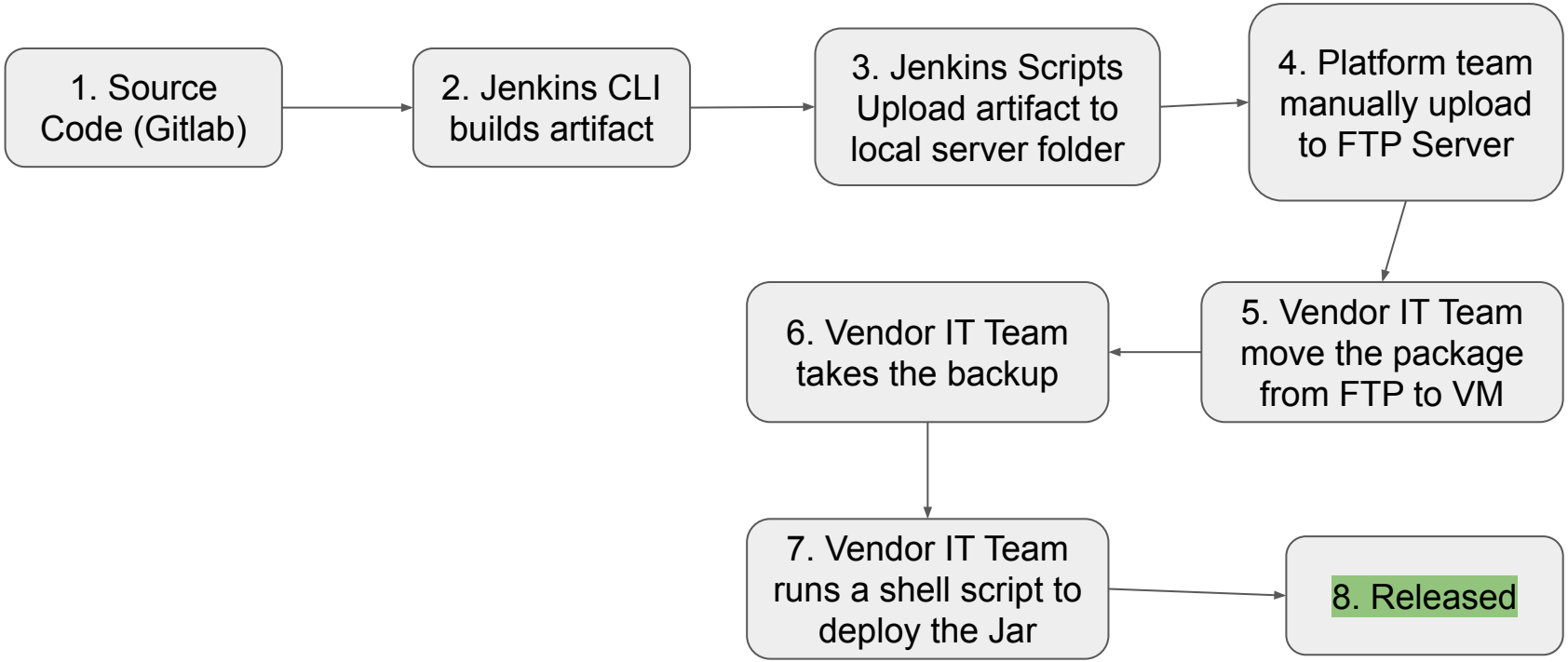
Module - 16:
CI/CD & Automation

- Amjad Hossain
18th Feb, 2026

Current Release Flow



Current Release Flow (??)



Questions:

1. How much automation we've right now using Jenkins?
2. How much automation we've in Prod environment using Shell Scripts?
3. How much are we allowed to automate in Production?

Let’s establish few rigid rules (not limited to)

1. There will be a Task Ticket for any change in source code

Let’s establish few rigid rules (not limited to)

1. There will be a Task Ticket for any change in source code
2. main branch of the repo will be locked for direct commit and push

Let’s establish few rigid rules (not limited to)

1. There will be a Task Ticket for any change in source code
2. main branch of the repo will be locked for direct commit and push
3. main branch is always releasable

Let’s establish few rigid rules (not limited to)

1. There will be a Task Ticket for any change in source code
2. main branch of the repo will be locked for direct commit and push
3. main branch is always releasable
4. Main branch is updateable only via Pull Request

Let’s establish few rigid rules (not limited to)

1. There will be a Task Ticket for any change in source code
2. main branch of the repo will be locked for direct commit and push
3. main branch is always releasable
4. Main branch is updateable only via Pull Request
5. Developer will work on feature branch

Let’s establish few rigid rules (not limited to)

1. There will be a Task Ticket for any change in source code
2. main branch of the repo will be locked for direct commit and push
3. main branch is always releasable
4. Main branch is updateable only via Pull Request
5. Developer will work on feature branch
6. Feature branch name must have Task Ticket ID as prefix with #

Let’s establish few rigid rules (not limited to)

1. There will be a Task Ticket for any change in source code
2. main branch of the repo will be locked for direct commit and push
3. main branch is always releasable
4. Main branch is updateable only via Pull Request
5. Developer will work on feature branch
6. Feature branch name must have Task Ticket ID as prefix with #
7. Each Pull Request will require at least 1 approver before merging

Let’s establish few rigid rules (not limited to)

1. There will be a Task Ticket for any change in source code
2. main branch of the repo will be locked for direct commit and push
3. main branch is always releasable
4. Main branch is updateable only via Pull Request
5. Developer will work on feature branch
6. Feature branch name must have Task Ticket ID as prefix with #
7. Each Pull Request will require at least 1 approver before merging
8. Pull Request must pass Code Linter

Let’s establish few rigid rules (not limited to)

1. There will be a Task Ticket for any change in source code
2. main branch of the repo will be locked for direct commit and push
3. main branch is always releasable
4. Main branch is updateable only via Pull Request
5. Developer will work on feature branch
6. Feature branch name must have Task Ticket ID as prefix with #
7. Each Pull Request will require at least 1 approver before merging
8. Pull Request must pass Code Linter
9. Pull Request must find unit tests that are not broken

Let’s establish few rigid rules (not limited to)

1. There will be a Task Ticket for any change in source code
2. main branch of the repo will be locked for direct commit and push
3. main branch is always releasable
4. Main branch is updateable only via Pull Request
5. Developer will work on feature branch
6. Feature branch name must have Task Ticket ID as prefix with #
7. Each Pull Request will require at least 1 approver before merging
8. Pull Request must pass Code Linter
9. Pull Request must find unit tests that are not broken
10. Each month Pull Request ask for extra 10% of unit test coverage (until 95%)

Git Flow to maintain

Branches

- `main` → always releasable, only via PR
- `feature/<ticket>-<short-desc>` → dev branches for new feature
- `bug-fix/<ticket>-<short-desc>` → dev branches for bug fixes
- `hot-fix/<ticket>-<short-desc>` → dev branches for hotfixes

Rules

- No direct commits to `main`
- Everything goes through PR into `main`
- Tag releases from `main` (e.g., `v1.8.0`)

Pull Request rules

PR policy (minimum viable governance)

- Require **≥ 1 approval**
- Require **successful CI pipeline** (nothing broken)
- Require **conversation resolved** (All comment/feedback must be solved)
- Require **up-to-date branch** before merge (no stale PR merges)

Optional (add later)

- Release PR requires one more Approver from Release Manager

Code linter scan

Goal: fail fast on style/quality problems.

Recommended linters (free)

- **Checkstyle** (style rules)
- **SpotBugs** (bug patterns)
- **PMD** (code quality rules)

Blocking rule

- If linter finds violations above threshold → pipeline fails → PR cannot merge.

Unit tests

Requirement

- Unit tests must run on every PR.

Policy

- Test failures should fail the build (tests failing means unsafe),
- but **coverage threshold is not enforced** initially (you can still publish coverage reports).

Implementation

- `mvn test`
- publish JUnit results
- publish coverage as report (non-blocking)

Future Plan:

- Each month increase the required coverage percentage by 10

Upload to FTP Server (automated by pipeline)

Replace the manual “platform team uploads to FTP” step with CI.

FTP directory convention

- `/releases/<app>/<version>/...`
- Example:
  - `/releases/banking-app/1.8.0+105/`
    - `banking-app_1.8.0_105_ab12cd34.zip`
    - `manifest.json`
    - `sha256sum.txt`
    - `release-notes.md`
    - `READY`

Critical control

- Pipeline uploads files first, then uploads `READY` last
- Your existing deployment script (or watcher) only acts when `READY` exists.

Watcher Script in Staging VM

1. A shell Script will check fixed location of FTP Server
2. The Script copy the release package from FTP to Staging VM (safe location)
3. Creates a backup of previous release
4. Unzip the new Release and move to right path
5. Deploy the new release
6. If any exception happens, roll-back to last release automatically
7. Run the script as a service with Cron Job

Thank you